

Artificial Intelligence — Exam-Checklist Worksheet

Problems, hints & solutions for each point of the ai_8020 checklist

Part IB — Cambridge CST — Paper 7

Cover each Solution box first. The backprop derivation and the A* optimality proof are the two “do-it-cold” set-pieces.

1 Neural networks

Warm-up 1. Why must a network’s activation function be non-linear?

Hint. What is a composition of linear maps?

Solution

A composition of linear maps is itself linear, so any number of linear layers collapses to one — it could only ever represent a linear boundary (and so couldn’t learn XOR). Non-linearity is what gives MLPs their expressive power.

Warm-up 2. For the sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$, express $\sigma'(x)$ in terms of σ .

Hint. Differentiate; the answer is famously self-referential.

Solution

$$\sigma'(x) = \sigma(x) (1 - \sigma(x)).$$

Exam-style. With error $E = \frac{1}{2} \sum_k (a_k - t_k)^2$, output activations $a_k = g(\text{in}_k)$ and $\text{in}_k = \sum_j w_{jk} a_j$, derive the gradient $\partial E / \partial w_{jk}$ for an output-layer weight and hence the update rule; then state the hidden-layer δ , and explain why a single perceptron cannot learn XOR.

Hint. Chain rule through a_k and in_k . Define $\delta_k = \partial E / \partial \text{in}_k$.

Solution

$$\frac{\partial E}{\partial w_{jk}} = \underbrace{\frac{\partial E}{\partial a_k}}_{(a_k - t_k)} \cdot \underbrace{\frac{\partial a_k}{\partial \text{in}_k}}_{g'(\text{in}_k)} \cdot \underbrace{\frac{\partial \text{in}_k}{\partial w_{jk}}}_{a_j} = a_j \delta_k, \quad \delta_k = (a_k - t_k) g'(\text{in}_k).$$

Update: $w_{jk} \leftarrow w_{jk} - \eta a_j \delta_k$. **Hidden unit j :** $\delta_j = g'(\text{in}_j) \sum_k w_{jk} \delta_k$ (sum over the units j feeds), and $\Delta w_{ij} = -\eta a_i \delta_j$. One backward sweep of δ ’s yields every gradient.

XOR: its four points $(0,0) \rightarrow 0, (1,1) \rightarrow 0, (0,1) \rightarrow 1, (1,0) \rightarrow 1$ are *not linearly separable* — no single straight line puts both 1-outputs on one side — so a single linear-threshold unit cannot classify them. A hidden layer (composing two linear separators) can.

2 Heuristic search (A*)

Warm-up 1. Write $f(n)$ and define an *admissible* heuristic.

Hint. g is cost-so-far.

Solution

$f(n) = g(n) + h(n)$; h is admissible if $h(n) \leq h^*(n)$ for all n (never overestimates the true remaining cost).

Warm-up 2. Define a *consistent* (monotone) heuristic.

Hint. A triangle-inequality on the heuristic.

Solution

$h(n) \leq c(n, a, n') + h(n')$ for every edge $n \rightarrow n'$. Consistency implies admissibility, and makes f non-decreasing along any path.

Exam-style. State which property guarantees A* optimality for tree-search vs graph-search, and *prove* that tree-search A* with an admissible heuristic returns an optimal solution.

Hint. Suppose a suboptimal goal is about to be expanded; compare its f to that of a frontier node on the optimal path.

Solution

Tree-search A* is optimal if h is *admissible*; **graph-search** A* is optimal if h is *consistent* (else you may re-expand nodes / accept a worse path).

Proof (tree-search, admissible h). Let C^* be the optimal cost and suppose a suboptimal goal G_2 (with $g(G_2) > C^*$) is about to be expanded. Since $h(G_2) = 0$, $f(G_2) = g(G_2) > C^*$. Let n be a node on an optimal path to the optimal goal that is still on the frontier. By admissibility, $f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$. Hence $f(n) \leq C^* < f(G_2)$, so A* would expand n before G_2 — contradicting that G_2 was chosen. Therefore no suboptimal goal is ever expanded. $\square$

Also: IDA*/RBFS give A*-style optimality in *linear* memory; greedy hill-climbing on h alone is not optimal (local optima, plateaux).

3 Constraint satisfaction (CSP)

Warm-up 1. After assigning a variable, what does *forward checking* do?

Hint. Look at unassigned neighbours.

Solution

It removes from each unassigned neighbour's domain any value inconsistent with the new assignment; if a domain becomes empty, backtrack immediately.

Warm-up 2. In AC-3, when is a value removed from $D(X_i)$ while revising arc (X_i, X_j) ?

Hint. "Support".

Solution

When that value has *no support* — no value in $D(X_j)$ satisfies the constraint between X_i and X_j with it.

Exam-style. Solve, by backtracking with forward checking, the CSP: variables A, B, C , each domain $\{r, g\}$, constraints $A \neq B$, $B \neq C$, $A \neq C$. Show the search and conclude.

Hint. This is 2-colouring a triangle. Watch a domain empty out.

Solution

- Assign $A = r$. Forward-check: $A \neq B \Rightarrow$ remove r from B ($D(B) = \{g\}$); $A \neq C \Rightarrow D(C) = \{g\}$.
- Assign $B = g$. Forward-check $B \neq C \Rightarrow$ remove g from $C \Rightarrow D(C) = \emptyset$. **Dead end** — backtrack. B has no other value $\Rightarrow$ backtrack to A .
- Assign $A = g$: symmetric, same empty-domain failure.

No solution — a triangle (odd cycle) is not 2-colourable. Forward checking caught the failure as soon as C 's domain emptied, without a full assignment.

4 Planning

Warm-up 1. What are the three components of a STRIPS action?

Hint. What must hold; what becomes true; what becomes false.

Solution

Precondition, add-list, delete-list. (Applying a in state s : $s' = (s \setminus \text{Del}) \cup \text{Add.}$)

Warm-up 2. In a planning graph, when are two actions *mutex* by interference?

Hint. One undoes something the other needs/produces.

Solution

When one action deletes a precondition or an (add-)effect of the other.

Exam-style. “Have cake and eat it”: init $\{Have\}$; actions EAT (pre $Have$; add $Eaten$; del $Have$) and BAKE (pre $\neg Have$; add $Have$); goal $\{Have, Eaten\}$. Build the planning graph far enough to find a plan, explaining the mutex that forces expansion.

Hint. At the first action level only EAT (and persistence “no-ops”) apply. Check whether both goals appear non-mutex at the next state level.

Solution

$S_0 = \{Have, \neg Eaten\}$.

A_0 : EAT (pre $Have$) plus persistence no-ops for $Have$ and $\neg Eaten$.

$S_1 = \{Have, \neg Have, Eaten, \neg Eaten\}$. Both goal literals $Have$ and $Eaten$ are present — but they are **mutex**: $Eaten$ is produced only by EAT, which *deletes* $Have$, whereas $Have$ here comes from the persistence no-op; those two actions interfere. So the goal can't be extracted at level 1 $\Rightarrow$ expand again.

A_1 now also has BAKE (pre $\neg Have$, available after EAT); at S_2 , $Have$ (from BAKE) and $Eaten$ (persisted) are *no longer* mutex. Extract the plan: **Eat then Bake**. (The planning graph also gives an admissible heuristic: the first level at which all goals appear non-mutex lower-bounds the plan length.)

5 Lighter topics (search, games, logic)

Warm-up 1. Which uninformed search is complete and optimal for unit step-costs but uses $O(b^d)$ memory?

Hint. Explores level by level.

Solution

Breadth-first search. (Iterative deepening gets the same guarantees in $O(bd)$ memory.)

Warm-up 2. What is the best-case time complexity of minimax with α - β pruning, and what does it mean?

Hint. Compare to plain minimax $O(b^m)$.

Solution

$O(b^{m/2})$ with perfect move ordering — effectively *doubling* the depth searchable in the same time.

Exam-style. Trace α - β on this tree (root MAX; two MIN children; leaves left-to-right). B 's leaves: 3, 12, 8. C 's leaves: 2, ?, ?. Show the value returned and which leaves are pruned. Then state the difference between forward and backward chaining.

Hint. Evaluate B fully to set α ; then evaluate C 's first leaf and compare to α .

Solution

B (MIN) = $\min(3, 12, 8) = 3$, so at the root $\alpha = 3$. Now explore C (MIN): its first leaf is 2, so $C \leq 2$. Since $2 \leq \alpha = 3$, C can only *lower* the MIN and can never beat B — **prune C 's remaining leaves** (β -cutoff). Root = $\max(3, \leq 2) = \boxed{3}$, and the second and third leaves of C are never examined.

Chaining: *forward* chaining is **data-driven** — start from known facts, fire rules to derive new facts until the goal appears. *Backward* chaining is **goal-driven** — start from the goal, find rules that conclude it, and recursively try to establish their premises.

Re-derive backprop and the A^ optimality proof cold, and re-run the AC-3/forward-checking and α - β traces, before Paper 7.*